MALGONNE
Léo

Rapport d’activité Terraform

MALGONNE
Léo

Table des matières

Introduction 3
Architecture du projet 3
Terraform 4
VM 4
Blob storage 4
Ansible 5
API et Backend 6
Test des endpoints API 7
Conclusion 8
Annexes 8
- Déploiement de terraform 8
- Installation de NodeJS et npm 10
- Ajout des logiques pour les fichiers 10
- Installation d'ansible 11

MALGONNE
Léo

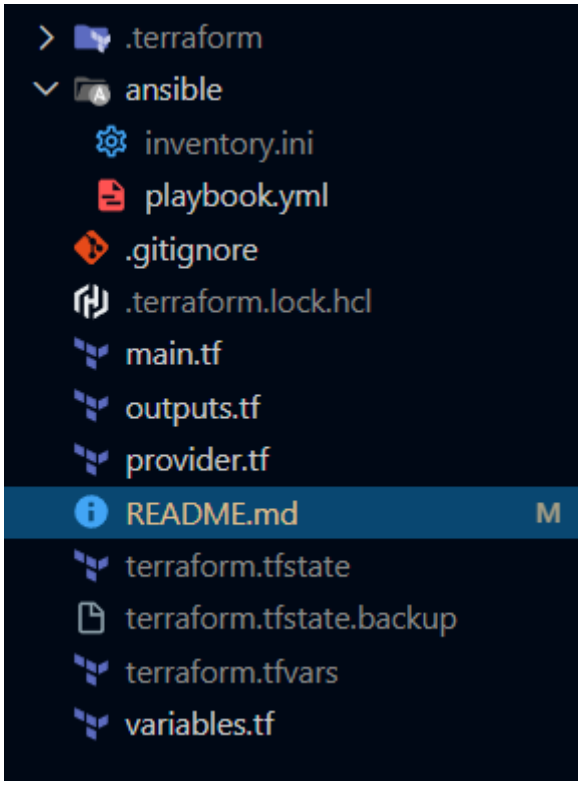
Introduction

Ce projet a pour objectif de déployer une infrastructure cloud complète sur Azure en utilisant une approche Infrastructure as Code**.**

Nous avons utilisé Terraform pour provisionner les ressources Azure (machine virtuelle, réseau, Blob Storage), Ansible pour automatiser la configuration de la VM, et Node.js pour exposer une API REST permettant d'effectuer des opérations CRUD sur des fichiers stockés dans un Azure Blob Storage.

L'ensemble du déploiement est automatisé, reproductible et versionnable, conformément aux bonnes pratiques DevOps.

Architecture du projet



L'infrastructure est entièrement définie via Terraform, organisée en quatre fichiers distincts :

- **provider.tf** : configure le provider Azure (azurerm) et le provider random pour la génération de noms uniques
- **variables.tf** : centralise toutes les variables pour rendre l'infrastructure dynamique (région, taille de VM, nom du projet...)
- **terraform.tfvars** : stocke les valeurs des variables, notamment les informations sensibles comme le chemin de la clé SSH
- **main.tf** : définit l'ensemble des ressources Azure à provisionner
- **outputs.tf** : expose les informations utiles après le déploiement (IP publique, URL de l'application, endpoint du Blob Storage)

MALGONNE
Léo

Terraform

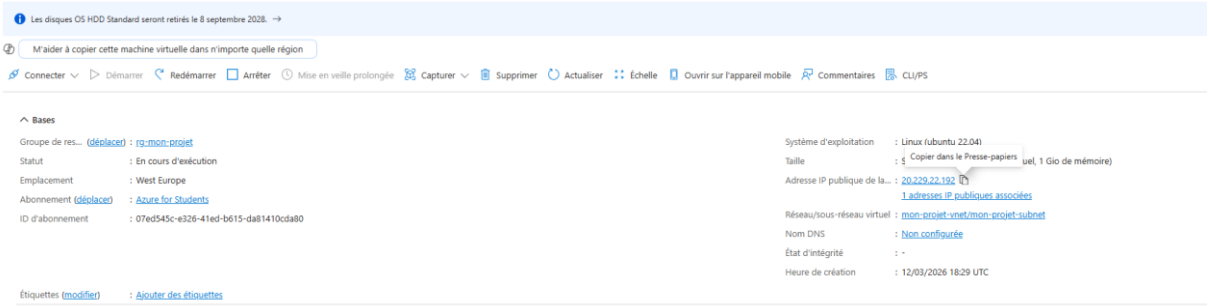
VM

La machine virtuelle est configurée avec les caractéristiques suivantes :

- Système d'exploitation : Ubuntu 22.04 LTS
- Taille : Standard_B1s (1 vCPU, 1 Go RAM)
- Authentification : clé SSH RSA 4096 bits
- IP publique : 20.229.22.192

L'accès à la VM utilise notamment les ports :

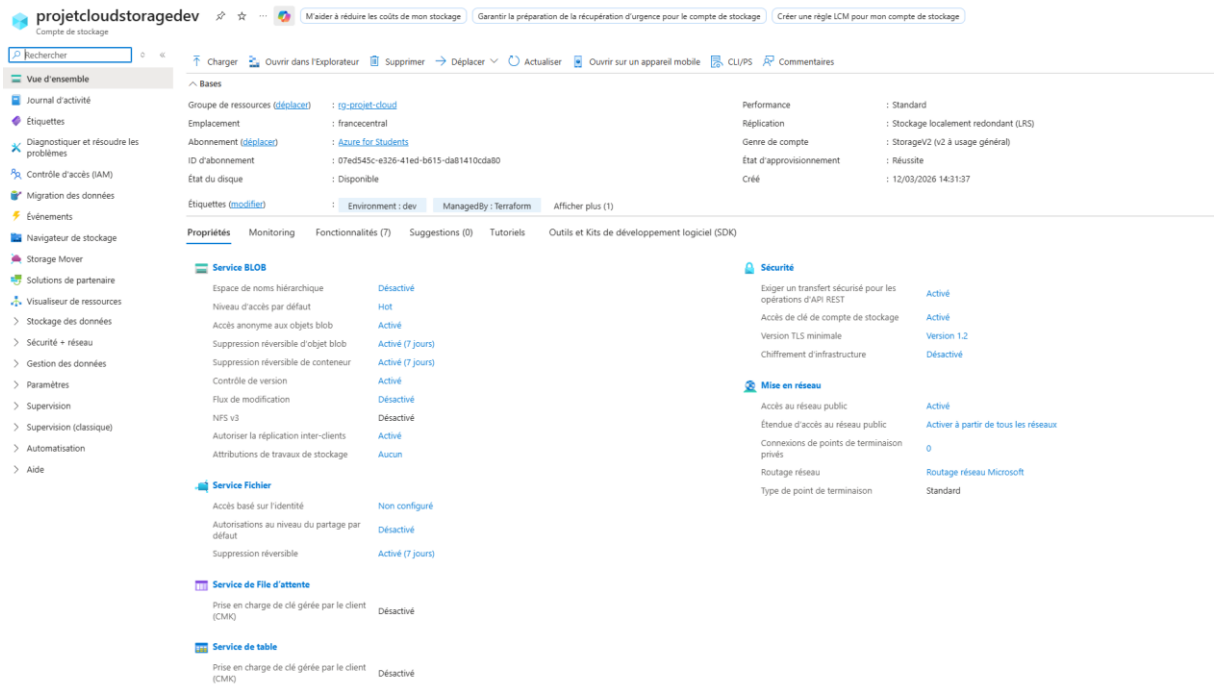
- 22 : accès SSH
- 8080 : accès à l'application Node.js



Blob storage

Le stockage est assuré par un Blob Storage composé d'un Storage Account et d'un container nommé fichiers-statiques. Le nom du Storage Account est généré dynamiquement via Terraform avec un suffixe aléatoire afin de garantir son unicité globale.

MALGONNE
Léo



Connexion SSH à la VM

```
PS C:\Users\agent\OneDrive\Efrei\3eme année\1er Semestre\Cloud\Projet terraform> ssh adminuser@20.16.110.229
See https://ubuntu.com/esm or run: sudo pro status

New release '20.04.6 LTS' available.
Run 'do-release-upgrade' to upgrade to it.


Last login: Thu Mar 12 10:06:41 2026 from 86.247.122.34
To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

adminuser@tp-vm:~$
```

Ansible

Ansible utilise des playbooks écrits en YAML décrivant les tâches à exécuter sur le serveur. L'avantage reste que l'on peu juste l'installer sur son pc en local et ne pas l'installer sur le serveur.

MALGONNE
Léo

```yaml
ansible > playbook.yml
---
- name: Configuration de la VM Azure
  hosts: vm
  become: yes

  vars:
    app_dir: /app
    app_port: 8080
    connection_string: "{{ lookup('env', 'AZURE_STORAGE_CONNECTION_STRING') }}"

  tasks:
    - name: Mettre à jour apt
      apt:
        update_cache: yes

    - name: Installer curl et git
      apt:
        name:
          - curl
          - git
        state: present

    - name: Installer NVM
      shell: |
        curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.39.7/install.sh | bash
      args:
        executable: /bin/bash
      become_user: adminuser
```

Résultat après avoir lancé ansible :

```
TASK [Installer PM2] ***********************************************************************************************
changed: [20.229.22.192]

TASK [Lancer l'app avec PM2] ***************************************************************************************
changed: [20.229.22.192]

TASK [Configurer PM2 au démarrage] *********************************************************************************
changed: [20.229.22.192]

PLAY RECAP *********************************************************************************************************
20.229.22.192              : ok=11   changed=7    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
```

API et Backend

Le backend est une API REST développée en Node.js utilisant le module http natif. Elle est déployée sur la VM Azure et accessible sur le port 8080 via l'IP publique.

ENDPOINTS :

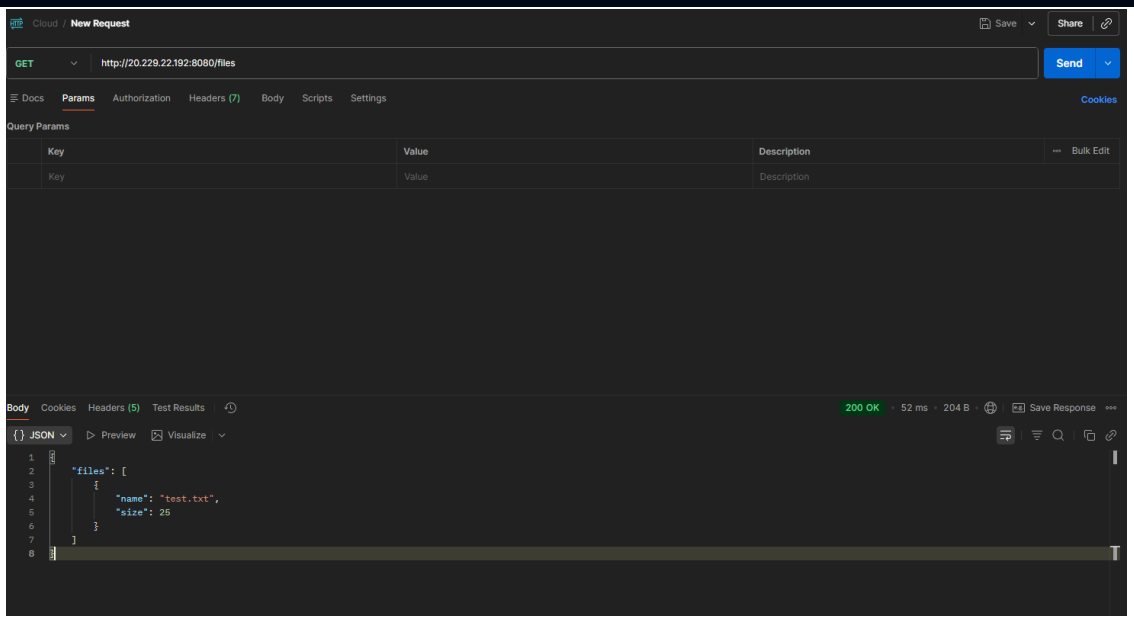
Méthode	Endpoint	Descriptif
/GET	/files	Liste de toute les fichiers
/GET	/files/ :nom	Donne les propriétés d'un fichier
/POST	/upload	Télécharger un fichier sur le serveur
/DELETE	/files/ :nom	Supprime un fichier

MALGONNE
Léo

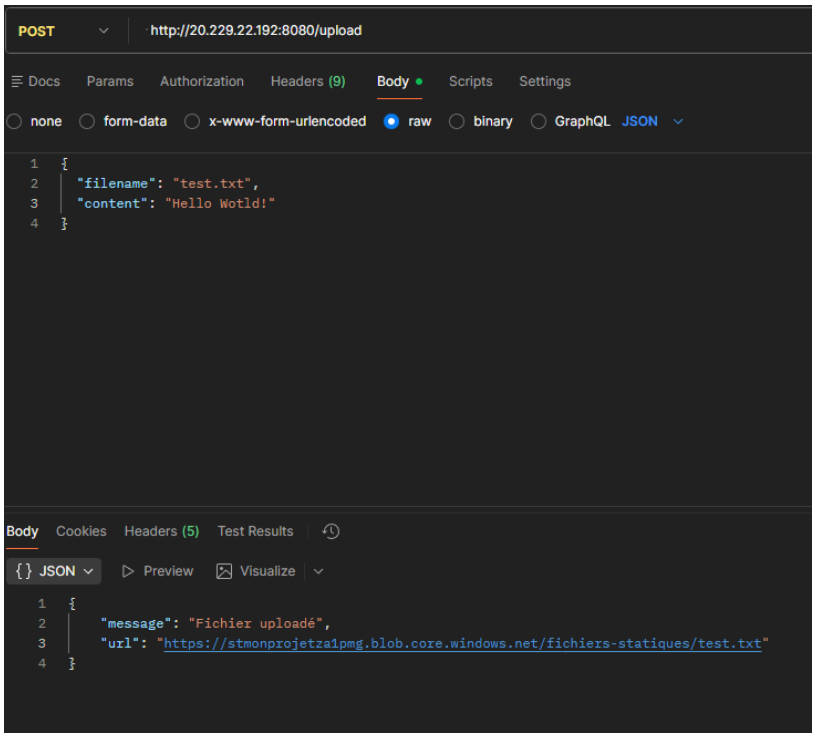
Test des endpoints API

Test du GET /files

```
PS C:\Users\agent\OneDrive\Efrei\3eme année\1er Semestre\Cloud\Projet-cloud> curl http://20.229.22.192:8080/files

StatusCode        : 200
StatusDescription : OK
Content           : {"files":[{"name":"test.txt","size":25}]}
RawContent        : HTTP/1.1 200 OK
                    Connection: keep-alive
                    Keep-Alive: timeout=5
                    Transfer-Encoding: chunked
                    Content-Type: application/json
                    Date: Thu, 12 Mar 2026 18:59:41 GMT
```

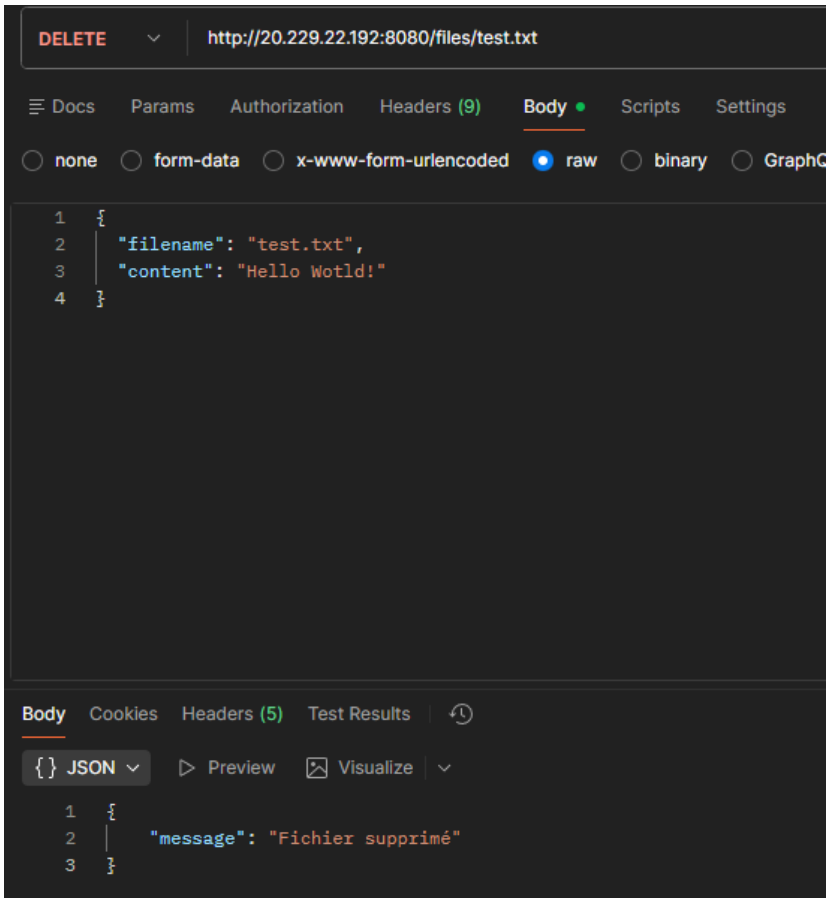


Test de POST /upload



MALGONNE
Léo

Test de DELETE /files/ :nom



Conclusion

Ce projet nous a permis de déployer une infrastructure cloud complète sur Azure en adoptant une approche Infrastructure as Code avec Terraform et Ansible. L'ensemble du déploiement est automatisé, de la création des ressources Azure jusqu'au lancement de l'application Node.js en tant que service.

La principale difficulté rencontrée la configuration de Terraform et celui de Ansible.

Des améliorations seraient envisageables, notamment l'ajout d'une base de données managée, la sécurisation de l'API avec JWT, et le stockage des secrets dans Azure Key Vault.

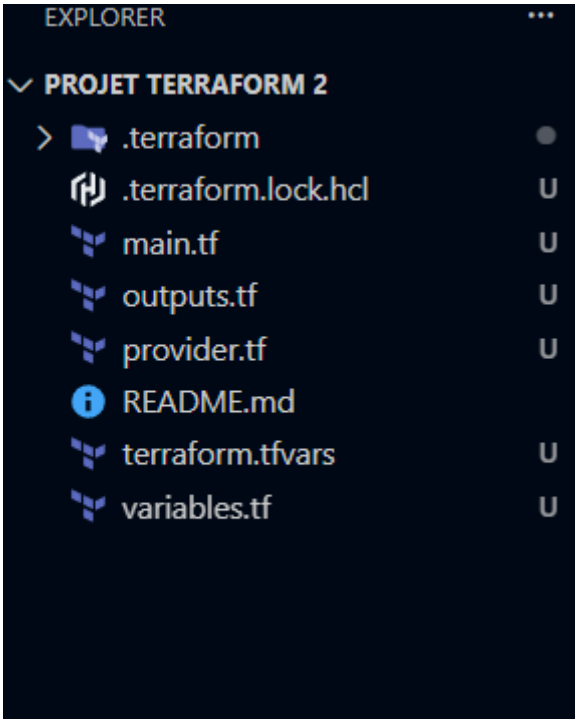
Annexes

Déploiement de terraform

Terraform init

MALGONNE
Léo

```
PS C:\Users\agent\OneDrive\Efrei\3eme année\1er Semestre\Cloud\Projet terraform 2> terraform init

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
```



Terraform plan

```
Changes to Outputs:
  + blob_container_name  = "fichiers-statiques"
  + ip_publique_vm       = (known after apply)
  + nom_resource_group   = "tp-terraform-rg"
  + storage_account_name = "tpstorage20456"

Note: You didn't use the -out option to save this plan, so Terraform can't guarantee to take exactly these actions if you run
"terraform apply" now.
```

Terraform apply

```
PS C:\Users\agent\OneDrive\Efrei\3eme année\1er Semestre\Cloud\Projet terraform> terraform apply
Terraform has compared your real infrastructure against your configuration and found no differences, so no changes are needed.

Apply complete! Resources: 0 added, 0 changed, 0 destroyed.

Outputs:

blob_container_name = "fichiers-statiques"
ip_publique_vm = "20.16.110.229"
nom_resource_group = "tp-terraform-rg"
storage_account_name = "tpstorage20456"
```

MALGONNE
Léo

Installation de NodeJS et npm

```
Now using node v16.20.2 (npm v8.19.4)
adminuser@mon-projet-vm:~$ nvm use 16
Now using node v16.20.2 (npm v8.19.4)
adminuser@mon-projet-vm:~$ node --version
v16.20.2
```

Ajout des logiques pour les fichiers

Ajout du fichier index.js

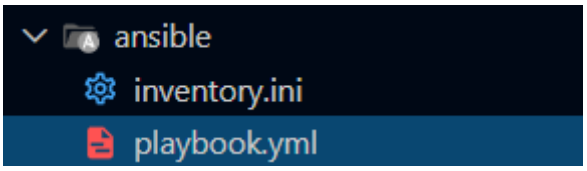
```
adminuser@mon-projet-vm:/app$ cat index.js
```

Exemple de l'index.js

```
  GNU nano 6.2
const http = require('http');
const { BlobServiceClient } = require('@azure/storage-blob');

const CONNECTION_STRING = "DefaultEndpointsProtocol=https;EndpointSuffix=core.windows.
const CONTAINER_NAME = "fichiers-statiques";

const blobServiceClient = BlobServiceClient.fromConnectionString(CONNECTION_STRING);
const containerClient = blobServiceClient.getContainerClient(CONTAINER_NAME);

async function uploadFile(filename, content) {
  const blockBlobClient = containerClient.getBlockBlobClient(filename);
  await blockBlobClient.upload(content, Buffer.byteLength(content));
  return blockBlobClient.url;
}

async function downloadFile(filename) {
  const blockBlobClient = containerClient.getBlockBlobClient(filename);
  const downloadResponse = await blockBlobClient.download(0);
  const chunks = [];
  for await (const chunk of downloadResponse.readableStreamBody) {
    chunks.push(chunk);
  }
  return Buffer.concat(chunks).toString();
}

async function listFiles() {
  const files = [];
  for await (const blob of containerClient.listBlobsFlat()) {
    files.push({
      name: blob.name,
      size: blob.properties.contentLength,
      lastModified: blob.properties.lastModified,
    });
  }
  return files;
}

async function deleteFile(filename) {
  const blockBlobClient = containerClient.getBlockBlobClient(filename);
  await blockBlobClient.delete();
}

const server = http.createServer(async (req, res) => {
  res.setHeader('Content-Type', 'application/json');

  try {
    if (req.method === 'GET' && req.url === '/files') {
      const files = await listFiles();
      res.writeHead(200);
      res.end(JSON.stringify({ files }));

    } else if (req.method === 'POST' && req.url === '/upload') {
      let body = '';
```

MALGONNE
Léo

Installation d'ansible

Sudo apt install ansible -y

Ajout de l'inventory.ini et du playbook